KasSigner User Guide

Wallets, backup/recovery, signing, mobile, and CoreS3 ownership

1. Security model and hardware

KasSigner keeps wallet spending authority offline. KasSee or another companion owns node queries, transaction construction, and broadcast. The signer independently parses the actual request and the user authorizes from the hardware display.

Target	Source support	Qualification
M5Stack CoreS3	Yes	Hardware-tested
CoreS3 Lite	Shared M5Stack target	Not separately HIL-tested
Waveshare ESP32-S3 variants	Yes	Require separate hardware qualification

> Consumer ESP32-S3 hardware has no secure element/tamper resistance. Use the threat model and release evidence appropriate to the value being protected.

2. Wallet lifecycle and slot types

- Up to 16 active slots can hold a 12/24-word mnemonic wallet, account XPrv, or one raw secp256k1 private key.
- Always Start Fresh keeps wallets RAM-only. Device-bound persistence is optional and encrypted using the device HMAC capability plus user credential.
- Name wallets so review/selection is unambiguous. Session-only restored wallets are not serialized to persistent storage.

3. Create and restore

- New mnemonic wallets may use 12 or 24 words and an optional BIP39 passphrase. Record recovery material offline before relying on the wallet.
- Restore Wallet opens: Words / SeedQR / SD / Advanced.
- Mnemonic restore order: import -> optional BIP39 passphrase -> wallet name -> Save Securely / Session Only. The words are not echoed back after entry.
- Account XPrv/raw-key restore skips BIP39 passphrase and continues to wallet name and storage choice.

4. Backup and Recovery menus

Menu	Purpose
Wallet -> Backup	View Words, SeedQR Backup, Encrypted SD Card, Advanced.
Backup -> Advanced	Compact/plain SeedQR, steganographic backup, XPrv backup, Export Key where applicable.
Wallet -> Recovery	SD Seed/XPrv, transaction/kpub/multisig/covenant restore types, and Import Raw Key.

After a successful backup, navigation returns to the Backup context that launched it rather than jumping to the wallet list.

5. KasSee pairing and addresses

- Export kpub/watch-only identity from the selected wallet and import it into KasSee.
- KasSee derives receive/change addresses and monitors UTXOs without the wallet spending private key.
- Verify high-value receive addresses on hardware when practical; use a private/custom node if address-query privacy matters.

6. Transaction signing and QR

- KasSee plans amounts, fees, UTXOs, change, network, and supported protocol context.
- Animated QR includes frame progress plus previous/next and pause/play controls.
- KasSigner scans the complete PSKT/PSKB or KSPT v4 request and independently reviews it.
- For multisig, hardware verifies descriptor-backed P2SH change before excluding it from Send.
- KasSee merges partial multisig signatures and broadcasts only after finalization.

7. Multisig, messages, and BIP85

- Multisig supports deterministic sorted participant keys, descriptor storage/import, receive/change derivation, co-signer relay, and final broadcast.
- Human-readable message signing remains domain-separated; generic wallet-key caller-selected hash signing is not exposed.
- BIP85 child mnemonics are available only from mnemonic wallet material and must be backed up according to their intended use.

8. Covenants and advanced protocols

KasSee builds the supported covenant, Private Swap, crowdfunding, oracle, and stealth workflows; KasSigner verifies/reviews and signs with the appropriate isolated authority. Unknown/opaque commitments receive stronger warnings. Rebuild abandoned historical transactions in current formats rather than relying on retired signing/session parsers.

9. CoreS3 Pop It! and owner firmware

Pop It! permanently enables ESP32-S3 Secure Boot v2. Before Pop It!, an owner may optionally enroll an independent RSA-3072 application-signing key at Settings -> Advanced -> Owner Firmware. If Pop It! happens first, owner enrollment is permanently closed.

- OWNERKEY.KAS contains the owner public digest and checksum only; the owner private key remains offline.
- After Pop It!, OWNERFW.BIN can be installed from SD. The vendor bootloader copies the staged image to the inactive OTA slot, verifies standard boot/anti-rollback policy and exact owner-key identity, then selects it only if every check succeeds.
- Official vendor applications remain supported. Owner authority does not replace the vendor second-stage bootloader.
- Owner enrollment write-protects both configured revoke controls. Lost owner keys do not block official updates, but compromised enrolled keys remain trusted.
- Development firmware simulates irreversible owner/Pop It! actions without eFuse writes.

10. Mobile apps

Android and iOS host the same KasSee Rust/WASM runtime in native platform/security shells. Both application paths have been tested. Publisher signing identities and keystores remain outside the repository; formal release evidence separately requires signed Release builds and physical-device smoke.

11. Build and release checklist

- Development: use make test for the fast contributor suite and make qa for the authoritative non-hardware suite.
- Physical qualification: use make test-hardware, make workflow-e2e, and make workflow-hil on the intended board.
- Release artifacts: use make release, then make release-readiness with the external signed evidence.
- Owner application: make owner-firmware OWNER_KEY=/secure/offline/path/owner.pem; never commit the owner private key or generated build artifacts.
- Before irreversible eFuse changes, follow the eFuse runbook and prove the complete sequence on sacrificial hardware.